

A High-Fidelity London Stock Exchange Simulation: A Technical Blueprint

Dimitris P. Skliros

August 15, 2025

Contents

1	Introduction	2
2	Core Platform Mechanics	2
2.1	Trading Day Lifecycle	2
2.2	Order Types and Validity	3
2.3	Order Matching Engine and the CLOB	3
2.4	Market Data Dissemination	3
3	SETS: The Stock Exchange Electronic Trading Service	4
3.1	Opening and Closing Auctions	4
3.2	Continuous Trading	4
4	Distinguishing Trading Types	4
4.1	Manual Trading	5
4.2	Algorithmic Trading	5
4.3	High-Frequency Trading (HFT)	5
5	Implementation Plan	5
5.1	apps/order_management/clob_engine.py	6
5.2	apps/trading_simulation/services.py	10
5.3	apps/market_data/streaming/service.py	11
5.4	apps/core/websockets/connection_manager.py	13

1 Introduction

The development of financial market simulations presents a fundamental choice between two distinct paradigms. The first, rooted in financial mathematics, models asset price movements as a continuous-time stochastic process, often using frameworks like Geometric Brownian Motion (GBM) to generate probabilistic price paths. While valuable for certain types of risk analysis and option pricing, this approach fundamentally abstracts away the very mechanisms that create the price.

The second paradigm, demanded by the study of market microstructure, treats the exchange not as a source of random variables but as a complex, deterministic, rule-based system that processes discrete events—orders, cancellations, and trades—to produce market outcomes. A high-fidelity trading simulation, capable of accurately testing the behavior of sophisticated algorithmic strategies, must adhere to this latter, more rigorous model.

This document provides a definitive technical blueprint for constructing a high-fidelity Python simulation of the London Stock Exchange (LSE). The objective is to move beyond simplistic backtesters and create a digital twin of the exchange’s core operational logic. This requires a deep and nuanced understanding of the LSE’s market microstructure, from its official rulebooks to its specific technological implementations.

2 Core Platform Mechanics

To build a high-fidelity simulation of the London Stock Exchange, you’ll need to incorporate several key mechanics from the provided documents. Here’s a detailed breakdown of the essential components:

2.1 Trading Day Lifecycle

The LSE trading day is divided into distinct phases, each with its own set of rules. Your platform will need to model this lifecycle accurately. The primary trading service for liquid securities is the **Stock Exchange Electronic Trading Service (SETS)**, which has the following phases:

- **Opening Auction (07:50 - 08:00 BST):** During this phase, orders are accepted but not executed. The system calculates a provisional uncrossing price, which is the price at which the maximum number of shares will be traded.
- **Continuous Trading (08:00 - 16:30 BST):** This is the main trading phase where orders are matched in real-time based on **price-time priority**.
- **Closing Auction (16:30 - 16:35 BST):** Similar to the opening auction, orders are collected, and a closing price is determined to execute a final batch of trades.
- **Post-Close (16:35 - 17:15 BST):** In this phase, trades can be reported, but no automated matching occurs.

For less liquid securities, the **SETSqx (Stock Exchange Electronic Trading Service – quotes and crosses)** service is used, which features periodic auctions throughout the day instead of continuous trading.

2.2 Order Types and Validity

Your platform must support the full range of LSE order types and their validity conditions. The most important ones are:

- **Limit Order:** An order to buy or sell at a specific price or better.
- **Market Order:** An order to buy or sell at the best available price in the market.
- **Iceberg Order:** A large order that is divided into smaller, visible "tranches" to hide the total order size.
- **Stop and Stop Limit Orders:** Orders that are triggered when the price of a security reaches a specific level.

Each order must also have a **Time in Force (TIF)** condition, which determines how long the order remains active. Key TIF conditions include:

- **Day:** The order is valid for the trading day.
- **Immediate or Cancel (IOC):** The order is executed immediately, and any portion that cannot be filled is canceled.
- **Fill or Kill (FOK):** The entire order must be executed immediately, or it is canceled.
- **Good Till Date (GTD):** The order remains active until a specified date.

2.3 Order Matching Engine and the CLOB

The heart of your simulation will be the **Central Limit Order Book (CLOB)** and the matching engine. The LSE uses a **price-time priority** algorithm for matching orders:

1. **Price Priority:** Orders with a higher price (for buys) or a lower price (for sells) are given priority.
2. **Time Priority:** If two orders have the same price, the one that was entered first gets priority.

Your `clob_engine.py` will need to implement this logic precisely. It should maintain two sorted lists of orders: one for bids (buy orders) and one for asks (sell orders). When a new order is submitted, the engine must attempt to match it against the existing orders in the book.

2.4 Market Data Dissemination

A crucial aspect of the simulation is providing realistic market data to users. This includes:

- **Real-time Price Feeds:** Your platform should stream live price updates, order book changes, and trade executions to connected clients via WebSockets.
- **Market Depth:** The order book should display the aggregate volume of buy and sell orders at different price levels.
- **Trade Ticker:** A real-time feed of all executed trades, including price, volume, and time.

3 SETS: The Stock Exchange Electronic Trading Service

SETS is the LSE's flagship electronic order book, handling the most liquid securities. A high-fidelity simulation must replicate its mechanics in detail.

3.1 Opening and Closing Auctions

The opening and closing auctions are critical for price discovery. They operate on four key principles:

1. **Maximising Executable Volume:** The auction price is the one that results in the largest number of shares being traded.
2. **Minimising Surplus:** If multiple prices maximise volume, the one with the smallest surplus (unmatched shares) is chosen.
3. **Market Pressure:** If a surplus still exists, the price is adjusted based on market pressure. A surplus of buy orders pushes the price up, and a surplus of sell orders pushes it down.
4. **Reference Price:** As a final tie-breaker, the price closest to the previous day's closing price (for the opening auction) or the last automated trade price (for the closing auction) is selected.

3.2 Continuous Trading

During the continuous trading phase, orders are matched in real-time based on price-time priority. The key features to implement are:

- **Aggressive vs. Passive Orders:** An aggressive order is one that can be immediately matched against an existing order in the book (e.g., a buy order with a price greater than or equal to the best ask). A passive order is one that is added to the book to await a matching order.
- **Partial Fills:** If an incoming order can only be partially matched, the filled portion is executed, and the remainder is either added to the order book (for limit orders) or canceled (for IOC or FOK orders).
- **Iceberg Orders:** For iceberg orders, only the peak size is visible in the order book. When the peak is fully executed, a new peak is revealed until the total order quantity is filled.

4 Distinguishing Trading Types

A high-fidelity simulation must cater to different types of market participants. The three primary categories are manual traders, algorithmic traders, and high-frequency traders (HFT).

4.1 Manual Trading

Manual traders interact with the market through a graphical user interface (GUI). Their actions are characterized by human reaction times, and their decisions are based on a combination of fundamental analysis, technical analysis, and intuition.

Implementation Requirements:

- A rich, responsive web interface for order entry, portfolio management, and market data visualization.
- Real-time updates via WebSockets to provide a live trading experience.
- Latency is less critical, but a responsive UI is paramount.

4.2 Algorithmic Trading

Algorithmic traders use automated systems to execute trading strategies. These strategies can be based on a wide range of factors, including technical indicators, statistical arbitrage, or fundamental data.

Implementation Requirements:

- A well-documented REST API for programmatic order submission, cancellation, and retrieval of market data.
- WebSockets for real-time data feeds to drive the algorithms.
- The platform must handle a higher volume of API requests than manual trading.

4.3 High-Frequency Trading (HFT)

HFT is a subset of algorithmic trading characterized by extremely high speeds and order volumes. HFT strategies often rely on exploiting tiny, short-lived market inefficiencies.

Implementation Requirements:

- A low-latency API, potentially using a binary protocol like FIX (Financial Information eXchange) in a real-world scenario. In our simulation, this translates to a highly optimized REST API and WebSocket infrastructure.
- Co-location of the trading algorithm with the matching engine is a key concept in the real world. In our simulation, this means minimizing any network overhead between the algorithm and the CLOB.
- The platform must be able to handle thousands of orders and cancellations per second.

5 Implementation Plan

Based on your existing directory structure and the requirements outlined above, here is a plan for the new and modified classes and methods you'll need to implement.

5.1 apps/order_management/clob_engine.py

This file contains the core logic for the Central Limit Order Book, tightly integrated with Django models for persistence.

```
import logging
from typing import List, Tuple, Optional
from decimal import Decimal
from django.db import transaction, models
from django.utils import timezone
from .models import (
    SimulatedOrder, OrderBookLevel, OrderQueue, OrderBook,
    SimulatedTrade, Fill, OrderSide, OrderStatus
)

logger = logging.getLogger(__name__)

class CentralLimitOrderBook:
    def __init__(self, instrument):
        self.instrument = instrument
        self.order_book = instrument.order_book
        self.logger = logging.getLogger(f"{__name__}.CLOB.{instrument.real_ticker.symbol}")

    def submit_order(self, order: SimulatedOrder) -> Tuple[List[dict], int]:
        with transaction.atomic():
            if order.order_type == 'MARKET':
                return self._process_market_order(order)
            elif order.order_type == 'LIMIT':
                return self._process_limit_order(order)
            else:
                self.logger.warning(f"Unsupported order type: {order.order_type}")
                return [], order.quantity

    def _process_market_order(self, order: SimulatedOrder) -> Tuple[List[dict], int]:
        trades = []
        remaining_qty = order.quantity

        if order.side == OrderSide.BUY:
            target_levels = self.order_book.levels.filter(
                side=OrderSide.SELL
            ).exclude(
                quantity=0
            ).order_by('price')
        else:
            target_levels = self.order_book.levels.filter(
                side=OrderSide.BUY
            ).exclude(
                quantity=0
            ).order_by('-price')
```

```

for level in target_levels:
    if remaining_qty <= 0:
        break
    level_trades, remaining_qty = self._match_against_level(
        order, level, remaining_qty
    )
    trades.extend(level_trades)
    level.refresh_from_db()
    if level.quantity == 0:
        level.delete()

self._update_best_quotes()
self.logger.info(f"Market order {order.order_id}: {len(trades)} trades, {rema
return trades, remaining_qty

def _process_limit_order(self, order: SimulatedOrder) -> Tuple[List[dict], int]:
    trades = []
    remaining_qty = order.quantity

    if order.side == OrderSide.BUY:
        matching_levels = self.order_book.levels.filter(
            side=OrderSide.SELL,
            price__lte=order.price
        ).exclude(
            quantity=0
        ).order_by('price')
    else:
        matching_levels = self.order_book.levels.filter(
            side=OrderSide.BUY,
            price__gte=order.price
        ).exclude(
            quantity=0
        ).order_by('-price')

    for level in matching_levels:
        if remaining_qty <= 0:
            break
        level_trades, remaining_qty = self._match_against_level(
            order, level, remaining_qty
        )
        trades.extend(level_trades)
        level.refresh_from_db()
        if level.quantity == 0:
            level.delete()

    if remaining_qty > 0:
        self._add_to_order_book(order, remaining_qty)

    self._update_best_quotes()

```

```

self.logger.info(f"Limit order {order.order_id}: {len(trades)} trades, {remaini
return trades, remaining_qty

def _match_against_level(self, incoming_order: SimulatedOrder,
                        level: OrderBookLevel, quantity: int) -> Tuple[List[dict],
trades = []
remaining = quantity
order_queues = level.order_queue.filter(
    remaining_quantity__gt=0
).order_by('queue_position')

for queue_entry in order_queues:
    if remaining <= 0:
        break
    resting_order = queue_entry.order
    available_qty = min(remaining, queue_entry.remaining_quantity)
    trade_data = self._execute_trade_between_orders(
        incoming_order, resting_order, available_qty, level.price
    )
    trades.append(trade_data)
    queue_entry.remaining_quantity -= available_qty
    remaining -= available_qty
    if queue_entry.remaining_quantity == 0:
        queue_entry.delete()
    else:
        queue_entry.save()

total_quantity = level.order_queue.aggregate(
    total=models.Sum('remaining_quantity')
)['total'] or 0
level.quantity = total_quantity
level.order_count = level.order_queue.count()
level.save()
return trades, remaining

def _execute_trade_between_orders(self, aggressive_order: SimulatedOrder,
                                passive_order: SimulatedOrder,
                                quantity: int, price: Decimal) -> dict:
    buy_order = aggressive_order if aggressive_order.side == OrderSide.BUY else p
    sell_order = passive_order if aggressive_order.side == OrderSide.BUY else agg
    trade = SimulatedTrade.objects.create(
        exchange=self.instrument.exchange,
        instrument=self.instrument,
        buy_order=buy_order,
        sell_order=sell_order,
        quantity=quantity,
        price=price,
        is_aggressive=(aggressive_order.side == OrderSide.BUY)
    )

```

```

fee_rate = self.instrument.exchange.trading_fee_percentage
trade_value = quantity * price
fees = trade_value * fee_rate
Fill.objects.create(
    order=aggressive_order,
    trade=trade,
    quantity=quantity,
    price=price,
    fees=fees,
    liquidity_flag='TAKER'
)
Fill.objects.create(
    order=passive_order,
    trade=trade,
    quantity=quantity,
    price=price,
    fees=fees,
    liquidity_flag='MAKER'
)
for order in [aggressive_order, passive_order]:
    order.filled_quantity += quantity
    if order.filled_quantity >= order.quantity:
        order.status = OrderStatus.FILLED
        order.completion_timestamp = timezone.now()
    elif order.filled_quantity > 0:
        order.status = OrderStatus.PARTIALLY_FILLED
    if not order.first_fill_timestamp:
        order.first_fill_timestamp = timezone.now()
    order.last_fill_timestamp = timezone.now()
    order.save()
self.order_book.last_trade_price = price
self.order_book.last_trade_quantity = quantity
self.order_book.last_trade_timestamp = timezone.now()
self.order_book.daily_volume += quantity
self.order_book.daily_turnover += trade.notional_value
self.order_book.trade_count += 1
if not self.order_book.daily_high or price > self.order_book.daily_high:
    self.order_book.daily_high = price
if not self.order_book.daily_low or price < self.order_book.daily_low:
    self.order_book.daily_low = price
self.order_book.save()
self.logger.info(f"REAL TRADE: {quantity}@{price} between {aggressive_order.u
return {
    'trade_id': trade.trade_id,
    'quantity': quantity,
    'price': price,
    'aggressive_order': aggressive_order,
    'passive_order': passive_order
}

```

```

def _add_to_order_book(self, order: SimulatedOrder, quantity: int):
    level, created = OrderBookLevel.objects.get_or_create(
        order_book=self.order_book,
        side=order.side,
        price=order.price,
        defaults={'quantity': 0, 'order_count': 0}
    )
    max_position = level.order_queue.aggregate(
        max_pos=models.Max('queue_position')
    )['max_pos'] or 0
    OrderQueue.objects.create(
        level=level,
        order=order,
        remaining_quantity=quantity,
        queue_position=max_position + 1
    )
    level.quantity += quantity
    level.order_count += 1
    level.save()
    self.logger.info(f"Added order {order.order_id} to book: {quantity}@{order.pr

def _update_best_quotes(self):
    best_bid_level = self.order_book.levels.filter(
        side=OrderSide.BUY,
        quantity__gt=0
    ).order_by('-price').first()
    if best_bid_level:
        self.order_book.best_bid_price = best_bid_level.price
        self.order_book.best_bid_quantity = best_bid_level.quantity
    else:
        self.order_book.best_bid_price = None
        self.order_book.best_bid_quantity = 0
    best_ask_level = self.order_book.levels.filter(
        side=OrderSide.SELL,
        quantity__gt=0
    ).order_by('price').first()
    if best_ask_level:
        self.order_book.best_ask_price = best_ask_level.price
        self.order_book.best_ask_quantity = best_ask_level.quantity
    else:
        self.order_book.best_ask_price = None
        self.order_book.best_ask_quantity = 0
    self.order_book.save()

```

5.2 apps/trading_simulation/services.py

This service manages the overall trading simulation, including the trading day lifecycle and interactions with the CLOB engine.

```

import logging
from typing import Dict
from decimal import Decimal
from django.utils import timezone
from .models import SimulatedExchange, SimulatedInstrument, TradingSession
from apps.order_management.models import MatchingEngine, OrderBook, OrderBookLevel

logger = logging.getLogger(__name__)

class SimulatedExchangeService:
    def __init__(self):
        self.logger = logging.getLogger(f"{__name__}.{self.__class__.__name__}")

    def create_simulated_exchange(self, real_exchange, **kwargs) -> SimulatedExchange:
        sim_exchange = SimulatedExchange.objects.create(
            name=f"Simulated {real_exchange.name}",
            code=f"SIM_{real_exchange.code}",
            real_exchange=real_exchange,
            **kwargs
        )
        MatchingEngine.objects.create(
            exchange=sim_exchange,
            matching_algorithm='PRICE_TIME'
        )
        self.logger.info(f"Created simulated exchange: {sim_exchange.code}")
        return sim_exchange

    def add_instrument_to_exchange(self, sim_exchange: SimulatedExchange,
                                   real_ticker: Ticker, **kwargs) -> SimulatedInstrument:
        sim_instrument = SimulatedInstrument.objects.create(
            real_ticker=real_ticker,
            exchange=sim_exchange,
            **kwargs
        )
        OrderBook.objects.create(instrument=sim_instrument)
        self._initialize_instrument_pricing(sim_instrument)
        self.logger.info(f"Added instrument {real_ticker.symbol} to {sim_exchange.code}")
        return sim_instrument

    # ... (other methods from SimulatedExchangeService)

```

5.3 apps/market_data/streaming/service.py

This service is a sophisticated streaming engine responsible for orchestrating the real-time market data flow, featuring caching, failover, and a circuit breaker pattern.

```

import asyncio
import logging
from typing import Dict, List, Optional, Set

```

```

from decimal import Decimal
from django.core.cache import cache
from .common import MarketDataPoint, StreamStatus, DataQuality

logger = logging.getLogger(__name__)

class CircuitBreaker:
    def __init__(self, failure_threshold: int = 5, recovery_timeout: int = 60):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failure_count = 0
        self.last_failure_time = None
        self.state = "CLOSED"
        # ... (implementation from service.py)

class CacheManager:
    def __init__(self, cache_prefix: str = "streaming"):
        self.cache_prefix = cache_prefix
        self.default_ttl = 300
        self.high_frequency_ttl = 30
        self.metrics = {'hits': 0, 'misses': 0, 'sets': 0}
        # ... (implementation from service.py)

class DataFeedManager:
    def __init__(self):
        self.yfinance = YFinanceService()
        self.alpha_vantage = AlphaVantageService()
        self.circuit_breakers = {
            'yfinance': CircuitBreaker(failure_threshold=3, recovery_timeout=30),
            'alphavantage': CircuitBreaker(failure_threshold=2, recovery_timeout=60)
        }
        self.primary_source = 'yfinance'
        self.fallback_source = 'alphavantage'
        # ... (implementation from service.py)

class StreamingEngine:
    def __init__(self, max_symbols: int = 100, update_interval: float = 1.0):
        self.max_symbols = max_symbols
        self.update_interval = update_interval
        self.status = StreamStatus.STOPPED
        self.cache_manager = CacheManager()
        self.data_feed_manager = DataFeedManager()
        self.channel_layer = get_channel_layer()
        self.active_symbols: Set[str] = set()
        # ... (rest of the attributes)

    async def start(self) -> bool:
        if self.status == StreamStatus.RUNNING:
            return True

```

```

        self.status = StreamStatus.STARTING
        await self._initialize_ta_calculators()
        main_task = asyncio.create_task(self._main_streaming_loop())
        self._running_tasks.add(main_task)
        health_task = asyncio.create_task(self._health_monitor_loop())
        self._running_tasks.add(health_task)
        self.status = StreamStatus.RUNNING
        return True

    # ... (other methods from StreamingEngine)

streaming_engine = StreamingEngine()

```

5.4 apps/core/websockets/connection_manager.py

The ConnectionManager is a sophisticated class responsible for tracking individual Web-Socket connections, managing user sessions, and even queuing messages for offline users.

```

import asyncio
import json
import logging
from typing import Dict, Set, Optional
from collections import defaultdict, deque
from dataclasses import dataclass
from django.utils import timezone
from channels.layers import get_channel_layer

logger = logging.getLogger(__name__)

@dataclass
class ConnectionInfo:
    channel_name: str
    user_id: Optional[int]
    consumer_type: str
    connected_at: datetime
    last_activity: datetime
    # ... (implementation from connection_manager.py)

class ConnectionManager:
    def __init__(self):
        self.connections: Dict[str, ConnectionInfo] = {}
        self.user_connections: Dict[int, Set[str]] = defaultdict(set)
        # ... (rest of the attributes)

    async def register_connection(self, channel_name: str, user_id: Optional[int] = None,
                                consumer_type: str = "unknown") -> bool:
        self._ensure_background_tasks()
        now = timezone.now()
        if user_id:

```

```

        if len(self.user_connections.get(user_id, set())) >= self.max_connections:
            return False
    connection_info = ConnectionInfo(
        channel_name=channel_name,
        user_id=user_id,
        consumer_type=consumer_type,
        connected_at=now,
        last_activity=now
    )
    self.connections[channel_name] = connection_info
    if user_id:
        self.user_connections[user_id].add(channel_name)
        self.metrics.authenticated_connections += 1
        await self._deliver_queued_messages(user_id, channel_name)
    self.metrics.total_connections += 1
    self.metrics.active_connections += 1
    self.metrics.connections_by_type[consumer_type] += 1
    return True

    async def unregister_connection(self, channel_name: str):
        connection_info = self.connections.get(channel_name)
        if not connection_info:
            return
        if connection_info.user_id:
            self.user_connections[connection_info.user_id].discard(channel_name)
            if not self.user_connections[connection_info.user_id]:
                del self.user_connections[connection_info.user_id]
            self.metrics.authenticated_connections -= 1
        for group in connection_info.subscription_groups:
            self.connection_groups[group].discard(channel_name)
            if not self.connection_groups[group]:
                del self.connection_groups[group]
        self.metrics.active_connections -= 1
        self.metrics.connections_by_type[connection_info.consumer_type] -= 1
        duration = (timezone.now() - connection_info.connected_at).total_seconds()
        self.metrics.avg_connection_duration = (
            self.metrics.avg_connection_duration * 0.9 + duration * 0.1
        )
        del self.connections[channel_name]

    # ... (other methods from ConnectionManager)

connection_manager = ConnectionManager()

```